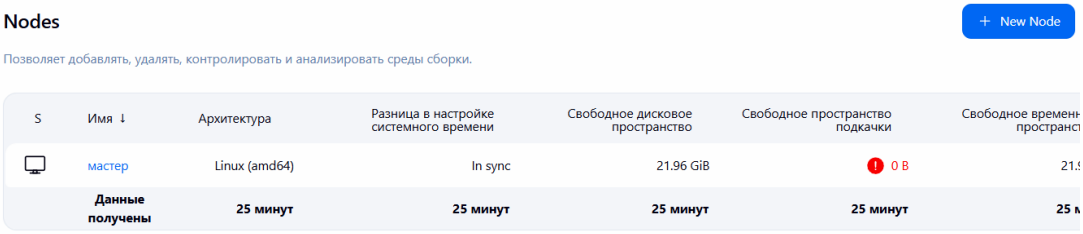
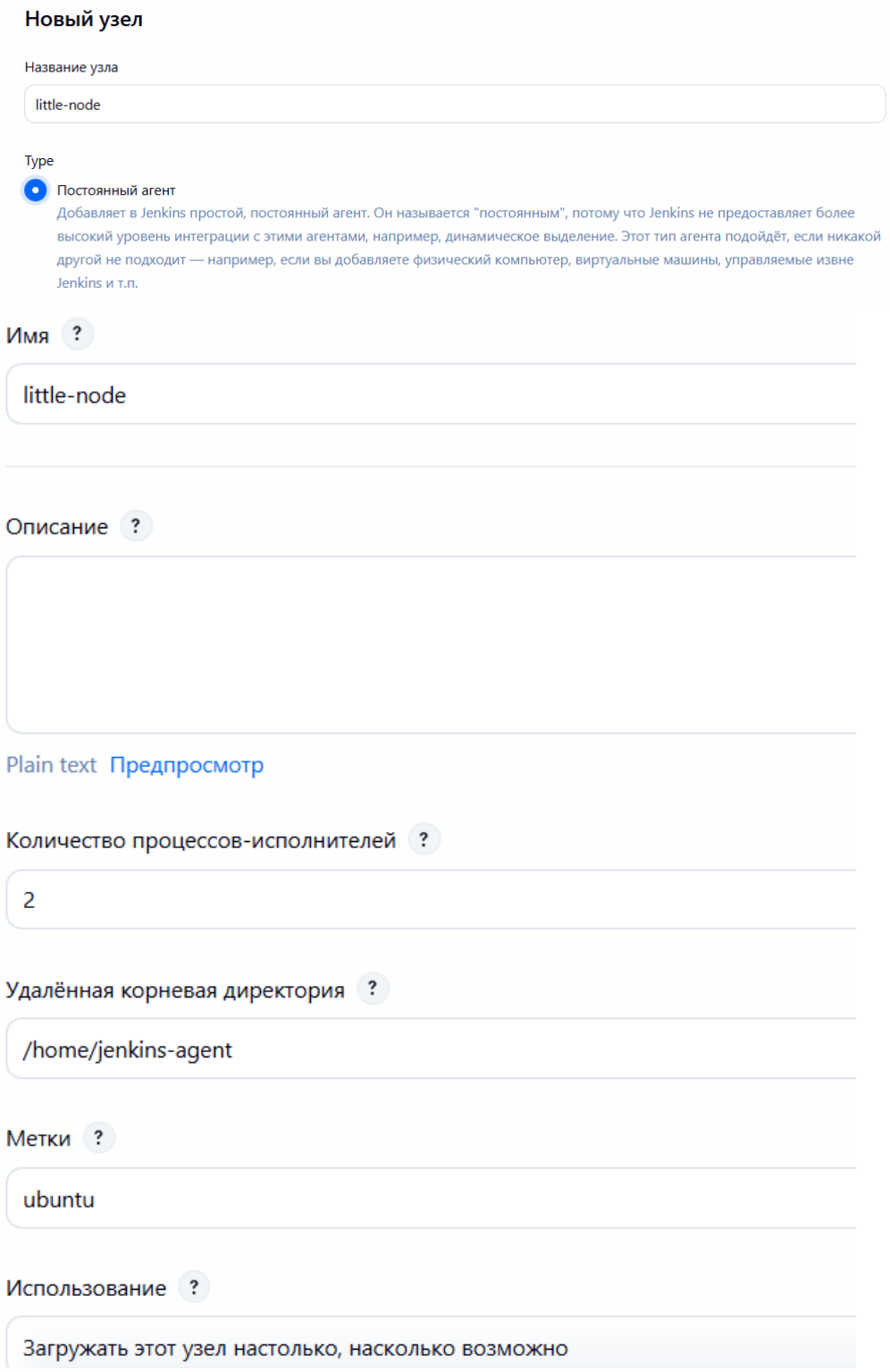
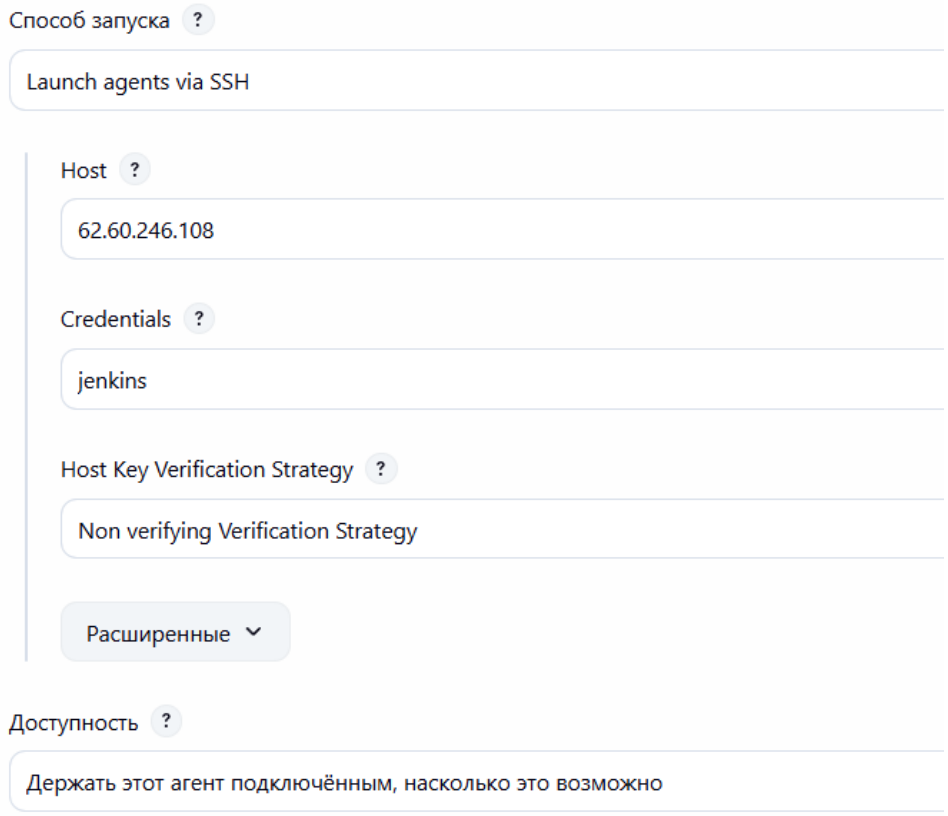
1. Перехожу в настройки jenkins – nodes и добавляю новую ноду (дженкинс агент):



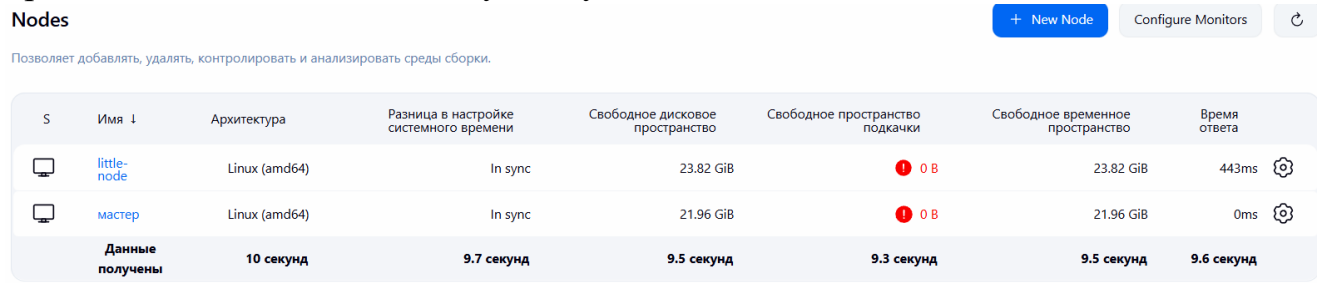
2. Настраиваю агента с названием little-node. Директорию указываю к которой у пользователя jenkins (создан был еще с первого ДЗ по пайплайну) есть разрешение и доступ, иначе будет ошибка:

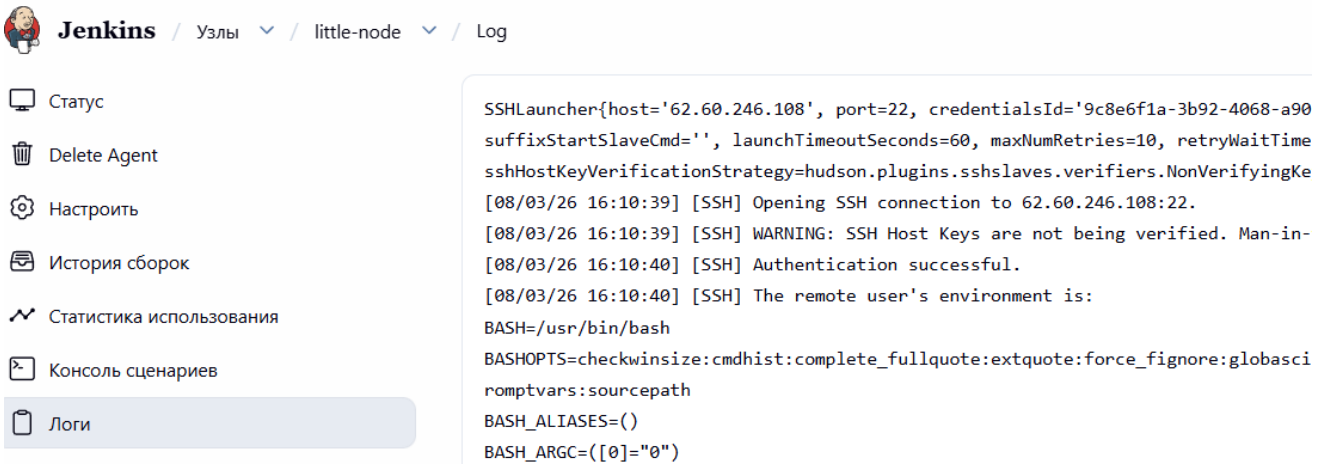


3. Обязательно выбираю пункт без проверки ключа, ведь если оставить по умолчанию, то при подключении он будет искать ключ в файле known_hosts и не найдет, даже если я подключусь напрямую с сервера.



4. Далее мы проверяем запуск агента, если все настроено верно, то итог будет таким. Если нет, то мы можем посмотреть логи попытки запуска, если провалимся в агента и слева будет пункт “Логи”:





5. Создаю новый item с названием HW-31. Пайплайн возьму из прошлого ДЗ, но теперь этапы сборки, тестирования и прочее кроме деплоя будет выполняться на агенте. А деплой будет происходить на сервере где развернут дженкинс. То есть все тоже самое, но меняется только айпи, пользователь сервера, также добавляю в строку agent метку созданного мною агента и создаю новый ssh ключ

```
pipeline {
    agent {
        label 'ubuntu'
    }
```

6. Результат работы пайплайна, как можно видеть контейнер был запущен на удаленном сервере и протестирован

```
anastasia@seigneur-puppy:~$ docker ps -a
CONTAINER ID   IMAGE                          COMMAND                  CREATED              STATUS                          PORTS     NAMES
c6d6064c4ff7   keshabyebye/maven-app:v01      "java -jar app.jar"      About a minute ago   Exited (0) About a minute ago             maven-app
```

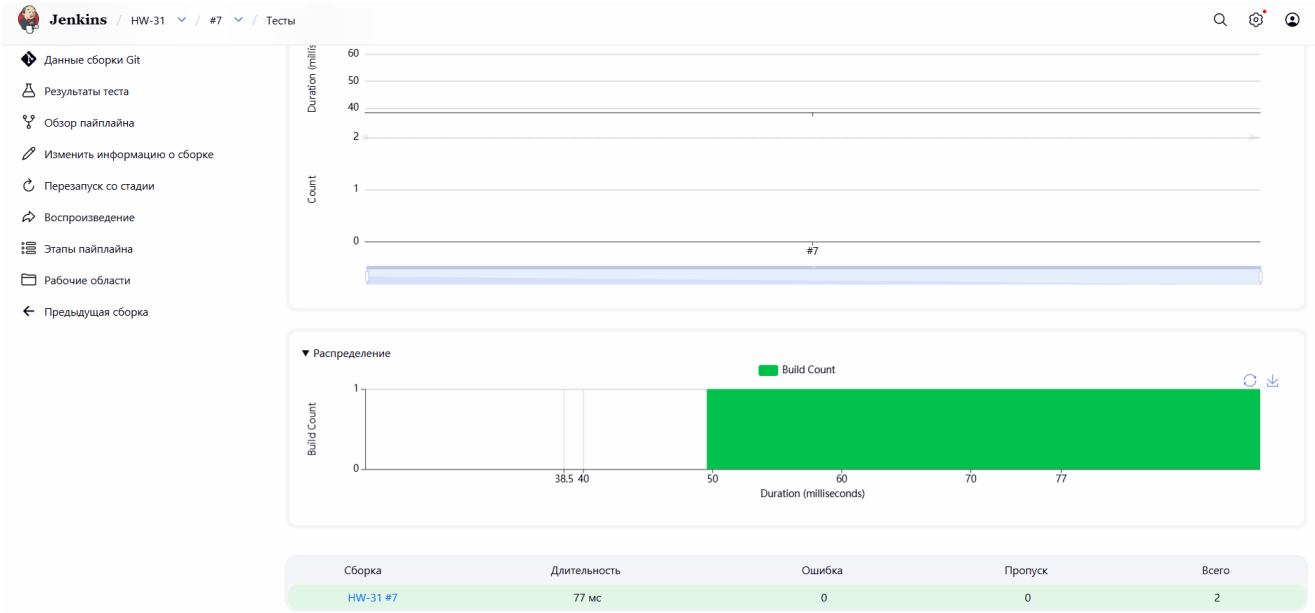
Вот лог:

```
Статус контейнера:
c6d6064c4ff7 keshabyebye/maven-app:v01 "java -jar app.jar" 7 секунд назад Завершено (0) 7 секунд назад maven-app
Логи приложения:
Hello World!
Приложение работает корректно!
```

7. В конструкцию post раздел always добавлю сбор отчета через Junit, чтобы при любом раскладе, будь то успех или провал

```
post {
    always {
        echo "Пайплайн завершен. Статус: ${currentBuild.result ?: 'SUCCESS'}"
        junit allowEmptyResults: true, testResults: "${APP_DIR}/target/surefire-reports/*.xml"
        cleanWs()
    }
```

8. Собранный отчет тестирования:



9. Способ защиты также присутствует, например авторизацию по логину и паролю, а также подключение к серверу по ssh:



10. Также отключаю повторяющиеся тесты, добавив к mvn clean package опцию –DskipTests. Итоговый пайплайн:

```
pipeline {
    agent {
        label 'ubuntu'
    }

    parameters {
        string(name: 'server', defaultValue: '193.233.216.149', description: 'IP сервера для запуска приложения')
        string(name: 'APP_NAME', defaultValue: 'maven-app', description: 'Название приложения')
        string(name: 'APP_VER', defaultValue: 'v01', description: 'Версия приложения')
        string(name: 'DOCKER_HUB_USER', defaultValue: 'keshabyebye', description: 'Логин на Docker Hub')
    }

    environment {
        WORKSPACE_DIR = "${env.WORKSPACE}"
        APP_DIR = 'simple-java-maven-app'
        DOCKER_IMAGE = "${params.DOCKER_HUB_USER}/${params.APP_NAME}:${params.APP_VER}"
    }
```

```
    stages {
        stage('Pre-check') {
            steps {
                script {
                    echo "Проверяем доступность сервера ${params.server}..."
                    sh "ping -c 2 ${params.server} || echo 'Ping не прошел, но пробуем
дальше'"
                }
            }
        }

        stage('Checkout') {
            steps {
                script {
                    echo "Клонируем репозиторий..."
                    sh "rm -rf ${APP_DIR}"
                    git branch: 'jenkins', url: 'https://github.com/SeigneurPuppy1/jenkins.git'
                }
            }
        }

        stage('Build') {
            steps {
                dir("${APP_DIR}") {
                    sh """
                    echo "=== СБОРКА MAVEN ПРОЕКТА ==="
                    mvn clean package -DskipTests
                    echo "JAR файлы:"
                    ls -la target/*.jar
                    """
                }
            }
        }

        stage('Test') {
            steps {
                script {
                    try {
                        echo "Запуск тестов..."
                        dir("${APP_DIR}") {
                            sh "mvn test"
                        }
                        echo "Тесты пройдены"
                    } catch (Exception e) {
```

```
                echo "Тесты не пройдены: ${e.message}"
                error("Тесты не пройдены")
            }
        }
    }
}

stage('Docker Build') {
    steps {
        script {
            echo "Создаем Docker образ..."
            dir("${APP_DIR}") {
                writeFile file: 'Dockerfile', text: """FROM eclipse-temurin:21-jre
WORKDIR /app
COPY target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
"""

                echo "Собираем образ: ${DOCKER_IMAGE}"
                sh """
                docker build -t ${DOCKER_IMAGE} .
                echo "Проверка образа:"
                docker images | grep ${params.APP_NAME}
                """
            }
        }
    }
}

stage('Docker Push to Hub') {
    steps {
        script {
            echo "Загружаем образ на Docker Hub..."
            withCredentials([usernamePassword(credentialsId: 'DOCKER_HUB',
usernameVariable: 'DH_USER', passwordVariable: 'DH_PASS')]) {
                sh """
                echo "Выполняем вход в Docker Hub..."
                echo "${DH_PASS}" | docker login -u "${DH_USER}" --password-
stdin

                echo "Пушим образ ${DOCKER_IMAGE}..."
                docker push ${DOCKER_IMAGE}
```

```
                    echo "Образ успешно загружен на Docker Hub!"
                    """
                }
            }
        }
    }

        stage('Deploy') {
        steps {
            script {
                echo "Развертывание на сервере ${params.server}..."

                withCredentials([usernamePassword(credentialsId: 'DOCKER_HUB',
usernameVariable: 'DH_USER', passwordVariable: 'DH_PASS')]) {
                    sshagent(credentials: ['ssh_key']) {
                        sh """
                        echo "Подключаемся к серверу и скачиваем образ с Docker
Hub..."

                        ssh -o StrictHostKeyChecking=no -o
UserKnownHostsFile=/dev/null \\
                            anastasia@${params.server} bash -s << 'EOF'
                            echo "Входим в Docker Hub на сервере..."
                            echo "${DH_PASS}" | docker login -u "${DH_USER}" --
password-stdin

                            echo "Скачиваем образ ${DOCKER_IMAGE} с Docker Hub..."
                            if ! docker pull ${DOCKER_IMAGE}; then
                                echo "ОШИБКА: Не удалось скачать образ с Docker Hub!"
                                exit 1
                            fi

                            echo "Останавливаем и удаляем старый контейнер (если
есть)..."
                            docker stop ${params.APP_NAME} 2>/dev/null || true
                            docker rm ${params.APP_NAME} 2>/dev/null || true

                            echo "Запускаем приложение..."
                            docker run \\
                                --name ${params.APP_NAME} \\
                                ${DOCKER_IMAGE}

                            echo "✅ Приложение выполнило свою задачу и
завершилось."
```

```
                    echo "Выходим из Docker Hub (очистка)..."
                    docker logout
EOF
                """
            }
        }
    }
}

stage('Verify') {
    steps {
        script {
            try {
                echo "Проверяем работу приложения..."
                sshagent(credentials: ['ssh_key']) {
                    sh """
                    ssh -o StrictHostKeyChecking=no -o
UserKnownHostsFile=/dev/null \\
                        anastasia@${params.server} bash -s << 'EOF'
                        sleep 5

                        echo "Статус контейнера:"
                        docker ps -a | grep ${params.APP_NAME} || exit 1

                        echo "Логи приложения:"
                        docker logs ${params.APP_NAME} || true

                        if docker logs ${params.APP_NAME} 2>&1 | grep -q "Hello
World"; then
                            echo "Приложение работает корректно!"
                            exit 0
                        else
                            echo "Приложение запущено, но вывод отличается"
                            exit 0
                        fi
EOF
                    """
                }
            } catch (Exception e) {
                echo "Ошибка проверки: ${e.message}"
                currentBuild.result = 'UNSTABLE'
            }
        }
```

```
            }
        }
    }

    post {
        always {
            echo "Пайплайн завершен. Статус: ${currentBuild.result ?: 'SUCCESS'}"
            junit allowEmptyResults: true, testResults: "${APP_DIR}/target/surefire-
reports/*.xml"
            cleanWs()
        }
        success {
            echo "Все этапы выполнены успешно!"
        }
        failure {
            script {
                echo "Пайплайн завершился с ошибками! Отправляю уведомление..."
                sh """
                MESSAGE="[ОШИБКА] Сборка \${JOB_NAME}
#\${BUILD_NUMBER}! Ссылка: \${BUILD_URL}"
                curl -s -X POST "https://api.vk.com/method/messages.send" \\
                  -d "peer_id=366737789" \\
                  -d "message=\${MESSAGE}" \\
                  -d "random_id=\$(date +%s)" \\
                  -d "access_token=${VK_ACCESS_TOKEN}" \\
                  -d "v=5.199" > /dev/null
                echo "Уведомление отправлено."
                """
            }
        }
    }
}
```